

Clase 018 — Git y control de versiones para profesionales de seguridad

Parte: **0 — Fundamentos y prerequisites** · Fuente: *Chacon & Straub, Pro Git* 🕒 Duración estimada: **100 min** · Nivel: **Fundamentos**

🎯 Objetivo

Usar Git con soltura para versionar herramientas, notas y hallazgos, colaborar y, sobre todo, evitar filtrar secretos en el historial, un error de seguridad extremadamente común. Al terminar manejarás el flujo básico, ramas, resolución de conflictos y prácticas de higiene de secretos.

📖 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Ejecutar** el flujo básico: init, add, commit, log, diff.
- 2. **Trabajar** con ramas, merge y resolución de conflictos.
- 3. **Sincronizar** con repositorios remotos (clone, push, pull).
- 4. **Prevenir** la fuga de secretos con `.gitignore` y buenas prácticas.
- 5. **Auditar** un repositorio en busca de secretos en el historial.

📖 Temas

#	Tema	Por qué importa
1	Modelo de Git	Snapshots, no diffs
2	Flujo básico	add/commit/log
3	Ramas y merge	Trabajo paralelo
4	Conflictos	Resolverlos sin romper nada
5	Remotos	Colaboración y respaldo
6	<code>.gitignore</code>	Evitar subir lo que no debe
7	Secretos en el historial	El error clásico de seguridad
8	Escaneo de secretos	gitleaks, trufflehog

📖 Definiciones y características

- **Commit**: instantánea del proyecto con un hash único. Clave: Git guarda snapshots completos, no diferencias.
- **Rama (branch)**: puntero móvil a una línea de desarrollo. Clave: aísla trabajo sin afectar `main`.

- **Merge:** integra una rama en otra. Clave: puede generar conflictos que hay que resolver a mano.
- **.gitignore:** lista de rutas que Git no debe rastrear. Clave: primera línea de defensa contra subir secretos/artefactos.
- **Historial inmutable:** reescribir el pasado (rebase, filter) cambia hashes. Clave: un secreto ya empujado no se borra con solo eliminarlo en un commit nuevo.
- **gitleaks/trufflehog:** escáneres de secretos en repos. Clave: detectan claves y tokens en todo el historial.

Herramientas y preparación

Instala Git y configúralo:

```
sudo apt install git
git config --global user.name "Tu Nombre"
git config --global user.email "tu@correo"
```

Para escaneo de secretos, instala **gitleaks** (<https://github.com/gitleaks/gitleaks>). Opcional: una cuenta en GitHub/GitLab para practicar remotos. **No** ejecute git en el repositorio del curso durante esta clase; usa un repo de práctica aparte.

Laboratorio guiado

1. **Crear un repo de práctica** (fuera del repo del curso):

```
bash mkdir ~/practica-git && cd ~/practica-git && git init
```

2. **Flujo básico:**

```
bash echo "# Notas" > README.md git add README.md && git commit -m "Primer commit" git log --
online
```

3. **Ramas y conflicto controlado.** Crea una rama, edita la misma línea en ambas y fuerza un conflicto al hacer merge; resuélvelo manualmente.
4. **.gitignore.** Añade patrones típicos de secretos y artefactos:

```
gitignore .env *.pem *.key venv/ __pycache__/
```

5. **Simular una fuga.** Crea un archivo `secreto.env` con una clave falsa, **añádalo por error** y haz commit. Comprueba que aparece en `git log -p`.
6. **Auditar con gitleaks:**

```
bash gitleaks detect --source . -v
```

Observa cómo detecta el secreto en el historial. 7. **Aprende la lección:** entiende que borrarlo en un commit nuevo **no** lo elimina del historial; haría falta reescribir el historial y **rotar** el secreto expuesto.

Ejercicios

1. Crea, cambia entre y fusiona tres ramas, documentando cada paso.
2. Provoca y resuelve un conflicto de merge en un archivo con dos ediciones.

3. Escribe un `.gitignore` completo para un proyecto Python de seguridad.
4. Usa `git log`, `git diff` y `git blame` para investigar quién y cuándo cambió una línea.
5. Ejecuta gitleaks sobre un repo con un secreto plantado y explica el hallazgo.
6. Investiga cómo eliminar un secreto del historial (`git filter-repo/BFG`) y por qué además hay que rotarlo.

Reto verificable

Crea un repositorio de práctica, simula la fuga de un secreto (clave ficticia) en un commit, détéctalo con gitleaks, y documenta el procedimiento correcto de remediación: eliminación del historial y rotación de la credencial. Añade un `.gitignore` que habría prevenido la fuga.

Criterio de aceptación: gitleaks detecta el secreto plantado en el historial; tu `.gitignore` incluye el patrón que lo habría bloqueado; y tu documento explica por qué "borrar el archivo en un commit posterior" es insuficiente y qué se hace en su lugar. Reproducible en un repo limpio.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Secreto ya empujado a remoto	Rotar la credencial de inmediato; reescribir historial no basta si alguien ya lo clonó.
<code>.gitignore</code> no ignora un archivo	Ya estaba rastreado. Ejecuta <code>git rm --cached archivo</code> .
Conflicto de merge sin resolver	Editar los marcadores <code><<<<<<</code> , <code>=====</code> , <code>>>>>>></code> y hacer commit.
<code>detached HEAD</code>	Hiciste checkout a un commit, no a una rama. Crea una rama desde ahí.
Push rechazado (non-fast-forward)	El remoto avanzó. Haz <code>git pull --rebase</code> y resuelve.

Preguntas frecuentes

? ¿Por qué es tan grave subir un secreto a Git? Porque el historial es distribuido y persistente: cualquiera que clonara el repo lo tiene, y bots escanean GitHub en segundos. La única respuesta segura es **rotar** el secreto.

? ¿`git rm` borra un secreto del historial? No: lo quita del árbol actual, pero sigue en commits anteriores. Hay que reescribir el historial (`filter-repo/BFG`) y rotar la clave.

? ¿Necesito ramas para trabajar solo? No es obligatorio, pero las ramas te permiten experimentar (por ejemplo, un exploit peligroso) sin ensuciar `main`. Es buena higiene.

? ¿gitleaks en cada commit? Sí, integrarlo como *pre-commit hook* evita fugas antes de que ocurran. Prevención mejor que remediación.

Referencias

- Chacon & Straub, *Pro Git* (gratis) — <https://git-scm.com/book>
- gitleaks — <https://github.com/gitleaks/gitleaks>
- GitHub: Removing sensitive data — <https://docs.github.com/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository>
- OWASP: Secrets Management Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

Siguiente clase

Clase 019 - Expresiones regulares para analisis de logs y datos